

PRACTICAL WEEK – 11

Session – 11

TASK 1: Develop a multithreaded counter application using POSIX threads (pthread create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

int counter = 0;

/* Thread function */
void *increment_counter(void *arg)
{
    int i;

    for (i = 0; i < INCREMENTS; i++)
    {
        counter++;
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int i;

    printf("Session 11 - Multithreaded Counter\n");
    printf("-----\n");

    printf("Number of threads : %d\n", NUM_THREADS);
    printf("Increments/thread : %d\n", INCREMENTS);
    printf("Expected counter : %d\n",
           NUM_THREADS * INCREMENTS);

    /* Create threads */
    for (i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                           increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1

int main()
{
    pthread_t threads[NUM_THREADS];
    int i;

    printf("Session 11 - Multithreaded Counter\n");
    printf("-----\n");

    printf("Number of threads : %d\n", NUM_THREADS);
    printf("Increments/thread : %d\n", INCREMENTS);
    printf("Expected counter : %d\n",
           NUM_THREADS * INCREMENTS);

    /* Create threads */
    for (i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                           increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }

    /* Wait for all threads */
    for (i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("\nActual counter : %d\n", counter);

    if (counter == NUM_THREADS * INCREMENTS)
    {
        printf("No race condition observed in this run.\n");
    }
    else
    {
        printf("Race condition detected!\n");
        printf("Some increments were lost.\n");
    }

    return 0;
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ nano session11_task1.c
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc session11_task1.c -o session11_task1 -pthread
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session11_task1
Session 11 - Multithreaded Counter
-----
Number of threads : 5
Increments/thread : 100000
Expected counter : 500000

Actual counter : 213484
Race condition detected!
Some increments were lost.
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ |
```

TASK 2: Modify the previous program using mutex locks (pthread mutex lock, pthread mutex_unlock) and compare the outputs with the race- condition version. Analyze the impact of synchronization.

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

int counter = 0;

/* Create mutex */
pthread_mutex_t lock;

/* Thread function */
void *increment_counter(void *arg)
{
    int i;

    for (i = 0; i < INCREMENTS; i++)
    {
        pthread_mutex_lock(&lock);

        counter++;

        pthread_mutex_unlock(&lock);
    }

    return NULL;
}

int main()
{
    pthread_t threads[NUM_THREADS];
    int i;

    printf("Session 11 - Mutex Synchronization\n");
    printf("-----\n");

    printf("Number of threads : %d\n", NUM_THREADS);
    printf("Increments/thread : %d\n", INCREMENTS);
    printf("Expected counter : %d\n",
           NUM_THREADS * INCREMENTS);
```

keerthi_priya@LAPTOP-OSGV × + ▾

GNU nano 8.7.1

```
printf("Expected counter : %d\n",
      NUM_THREADS * INCREMENTS);

/* Initialize mutex */
pthread_mutex_init(&lock, NULL);

/* Create threads */
for (i = 0; i < NUM_THREADS; i++)
{
    if (pthread_create(&threads[i], NULL,
                      increment_counter, NULL) != 0)
    {
        perror("pthread_create");
        return 1;
    }
}

/* Wait for all threads */
for (i = 0; i < NUM_THREADS; i++)
{
    pthread_join(threads[i], NULL);
}

printf("\nActual counter : %d\n", counter);

if (counter == NUM_THREADS * INCREMENTS)
{
    printf("Synchronization successful.\n");
    printf("No increments were lost.\n");
}
else
{
    printf("Synchronization failed.\n");
}

/* Destroy mutex */
pthread_mutex_destroy(&lock);

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ nano session11_task2.c
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ gcc session11_task2.c -o session11_task2 -pthread
keerthi_priya@LAPTOP-OSGVCE7A:/mnt/c/JavaPrograms$ ./session11_task2
Session 11 - Mutex Synchronization
-----
Number of threads : 5
Increments/thread : 100000
Expected counter : 500000

Actual counter : 500000
Synchronization successful.
No increments were lost.
```

